2026/9/3

LeetCode 数组学习笔记

学习内容概览

本笔记整理 `python/leetcode/数组` 目录中的 9 道题。重点不只是记录最终代码，也保留从暴力解法、错误思路到优化解法的推导过程。

题号	题目	核心方法	时间复杂度
11	盛最多水的容器	双指针	$O(n)$
13	罗马数字转整数	哈希映射、相邻比较	$O(n)$
15	三数之和	排序、双指针、去重	$O(n^2)$
42	接雨水	双指针、左右最大值	$O(n)$
128	最长连续序列	哈希集合	平均 $O(n)$
682	棒球比赛	栈	$O(n)$
896	单调数列	一次遍历、状态标记	$O(n)$
1041	困于环中的机器人	模拟、方向状态	$O(n)$
1275	找出井字棋的获胜者	模拟、分类计数	$O(n)$

11. 盛最多水的容器

问题分析

选择两条竖线作为容器边界，容量由两部分共同决定：

$$ S=\min(height[left], height[right])\times(right-left) $$

- 宽度由两个下标的距离决定。
- 高度由较短的竖线决定。
- 暴力枚举所有左右边界需要 $O(n^2)$，数据量大时会超时。

双指针思路

左右指针从数组两端开始。每次计算当前面积后，移动较短的一侧：

- 如果移动较高的一侧，宽度一定减小，而容器高度仍受较短边限制，不可能得到更优结果。
- 只有舍弃较短边，才有机会遇到更高的边，从而弥补宽度的损失。

```
from typing import List
```

```python
class Solution:
    def maxArea(self, height: List[int]) -> int:
        left, right = 0, len(height) - 1
        ans = 0

        while left < right:
            area = min(height[left], height[right]) * (right - left)
            ans = max(ans, area)

            if height[left] < height[right]:
                left += 1
            else:
                right -= 1

        return ans
```

- 时间复杂度：$O(n)$
- 空间复杂度：$O(1)$

> 双指针题的关键不是“同时使用两个指针”，而是证明每一步为什么可以排除一批不可能更优的答案。

---

13. 罗马数字转整数

> 源文件名是 `13_转罗马数字.py`，但其中实现的 `romanToInt` 实际对应“罗马数字转整数”。

问题分析

大多数罗马数字从左到右直接相加。特殊情况是较小数字出现在较大数字左侧，此时需要做减法，例如：

- `IV = 5 - 1 = 4`
- `IX = 10 - 1 = 9`
- `CM = 1000 - 100 = 900`

因此只需比较当前字符与右侧字符：

- 当前值小于右侧值：减去当前值。
- 否则：加上当前值。
- 最后一位没有右侧字符，直接加入答案。

```python
class Solution:
    def romanToInt(self, s: str) -> int:
        values = {
            "I": 1,
            "V": 5,
            "X": 10,
            "L": 50,
            "C": 100,
            "D": 500,
            "M": 1000,
```

```
        }

        ans = values[s[-1]]
        for i in range(len(s) - 1):
            if values[s[i]] < values[s[i + 1]]:
                ans -= values[s[i]]
            else:
                ans += values[s[i]]
        return ans
```

- 时间复杂度：$O(n)$
- 空间复杂度：$O(1)$，映射表大小固定

---

15. 三数之和

初始思路与问题

初版代码把数字分为正数、负数和零，再分别枚举不同组合。这种方法存在几个问题：

- 三层循环最坏达到 $O(n^3)$，会超时。
- 结果容易重复，需要额外去重。
- `list` 不能直接放入 `set`，因为列表是可变对象，不可哈希；元组可以放入集合。
- 分类讨论较多，边界情况复杂。

排序与双指针

先排序数组，再固定第一个数 `nums[i]`，在右侧区间使用左右指针寻找另外两个数。

1. 若三数之和为 0，记录答案，同时移动左右指针并跳过重复值。
2. 若三数之和小于 0，需要增大总和，因此左指针右移。
3. 若三数之和大于 0，需要减小总和，因此右指针左移。

```
class Solution:
    def threeSum(self, nums: list[int]) -> list[list[int]]:
        nums.sort()
        ans = []

        for i in range(len(nums) - 2):
            if nums[i] > 0:
                break
            if i > 0 and nums[i] == nums[i - 1]:
                continue

            left, right = i + 1, len(nums) - 1
            while left < right:
                total = nums[i] + nums[left] + nums[right]

                if total == 0:
                    ans.append([nums[i], nums[left], nums[right]])
                    left += 1
```

```
                    right -= 1

                    while left < right and nums[left] == nums[left - 1]:
                        left += 1
                    while left < right and nums[right] == nums[right + 1]:
                        right -= 1
                elif total < 0:
                    left += 1
                else:
                    right -= 1

        return ans
```

- 时间复杂度：排序 $O(n\log n)$，双指针枚举 $O(n^2)$，总体为 $O(n^2)$
- 空间复杂度：忽略排序实现与返回结果时为 $O(1)$

去重位置

三数之和最难的部分是去重：

- 固定数去重：进入本轮搜索前，比较 `nums[i]` 与 `nums[i - 1]`。
- 左右指针去重：找到答案并移动指针后，再跳过与刚才相同的数。
- 不能在搜索前随意跳过元素，否则可能漏掉合法组合。

---

42. 接雨水

每个位置能接多少水

位置 `i` 的积水量取决于它左侧最高柱和右侧最高柱中较矮的一根：

$$ water_i=\max(0,\min(leftMax_i,rightMax_i)-height_i) $$

暴力解法会为每个位置重复查找左右最大值，时间复杂度为 $O(n^2)$。

双指针优化

维护：

- `left_max`：左指针经过区域的最高柱。
- `right_max`：右指针经过区域的最高柱。

当 `left_max < right_max` 时，左侧位置的积水上限已经由 `left_max` 确定；反之处理右侧。

```
from typing import List


class Solution:
    def trap(self, height: List[int]) -> int:
        left, right = 0, len(height) - 1
        left_max = right_max = 0
```

```
        ans = 0

        while left < right:
            left_max = max(left_max, height[left])
            right_max = max(right_max, height[right])

            if left_max < right_max:
                ans += left_max - height[left]
                left += 1
            else:
                ans += right_max - height[right]
                right -= 1

        return ans
```

- 时间复杂度：$O(n)$
- 空间复杂度：$O(1)$

与"盛最多水的容器"的区别

- 盛水容器只选择两根柱子，计算一个整体矩形面积。
- 接雨水需要计算每个位置上方的积水，再累加所有位置。
- 两题都使用双指针，但指针移动的依据和状态含义不同。

---

128. 最长连续序列

初始思路与问题

排序后可以寻找连续数字，但排序本身需要 $O(n\log n)$。源代码中的初版还会在遇到第一处断点时直接返回，无法正确处理 `[1, 2, 6, 7, 8]` 这类存在多个连续区间的输入。

哈希集合思路

使用集合实现平均 $O(1)$ 查找。只从连续序列的起点开始向后统计：

- 如果 `num - 1` 存在，说明 `num` 不是起点，不需要重复搜索。
- 如果 `num - 1` 不存在，则不断检查 `num + 1`、`num + 2` 等数字。

```
from typing import List


class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        nums_set = set(nums)
        max_length = 0

        for num in nums_set:
            if num - 1 not in nums_set:
                current = num
                length = 1
```

```
                while current + 1 in nums_set:
                    current += 1
                    length += 1

                max_length = max(max_length, length)

        return max_length
```

- 时间复杂度：平均 $O(n)$。每个数字至多作为连续序列成员被访问一次。
- 空间复杂度：$O(n)$

> `set` 会去重，但不保证按照数值大小排序。此题依赖的是集合的快速查找，而不是集合的遍历顺序。

---

682. 棒球比赛

为什么使用栈

每次操作只依赖最近的有效得分：

- 整数：记录一次新得分。
- `+`：记录前两次有效得分之和。
- `D`：记录前一次有效得分的两倍。
- `C`：取消前一次有效得分。

源代码初版使用固定长度数组，并将取消的得分改成 0。这样“最近一次有效得分”的相对位置会被破坏。栈可以自然表示仍然有效的得分记录。

```
from typing import List


class Solution:
    def calPoints(self, operations: List[str]) -> int:
        scores = []

        for operation in operations:
            if operation == "+":
                scores.append(scores[-1] + scores[-2])
            elif operation == "D":
                scores.append(2 * scores[-1])
            elif operation == "C":
                scores.pop()
            else:
                scores.append(int(operation))

        return sum(scores)
```

- 时间复杂度：$O(n)$
- 空间复杂度：$O(n)$

> 涉及“最近一次有效记录”“撤销最近操作”时，应优先想到栈和负数索引。

---

896. 单调数列

两种方法

方法一是分别对数组升序、降序排序，再与原数组比较。这种写法直观，但排序需要 $O(n\log n)$ 时间和额外空间。

更好的方法是用两个布尔变量记录数组是否仍可能单调递增或单调递减：

```python
from typing import List


class Solution:
    def isMonotonic(self, nums: List[int]) -> bool:
        increasing = True
        decreasing = True

        for i in range(len(nums) - 1):
            if nums[i] < nums[i + 1]:
                decreasing = False
            if nums[i] > nums[i + 1]:
                increasing = False

        return increasing or decreasing
```

- 时间复杂度：$O(n)$
- 空间复杂度：$O(1)$

相邻元素相等不会破坏任何一种单调性，因此无需单独处理。

1041. 困于环中的机器人

状态模拟

机器人状态由坐标 `(x, y)` 和朝向共同决定。执行一轮指令后，只要满足以下任一条件，机器人就会被限制在有限区域中：

1. 回到原点。
2. 没有回到原点，但朝向不再是初始方向。

第二种情况下，重复执行相同指令会改变位移方向，最多执行四轮便会形成闭环。

角度处理的易错点

源代码初版只处理了角度大于等于 360 的情况，没有处理连续右转产生的负角度。使用 `% 360` 可以统一得到相对方向。

```python
class Solution:
    def isRobotBounded(self, instructions: str) -> bool:
        x = y = 0
        angle = 90

        for instruction in instructions:
            if instruction == "L":
                angle += 90
            elif instruction == "R":
                angle -= 90
            else:
                direction = angle % 360
                if direction == 0:
                    x += 1
                elif direction == 90:
                    y += 1
                elif direction == 180:
                    x -= 1
                else:
                    y -= 1

        return (x == 0 and y == 0) or angle % 360 != 90
```

- 时间复杂度：$O(n)$
- 空间复杂度：$O(1)$

1275. 找出井字棋的获胜者

问题分析

棋盘只有 3 行、3 列和 2 条对角线。每落一子，只需更新当前玩家对应的：

- 行计数。
- 列计数。
- 主对角线计数，条件为 `row == col`。
- 副对角线计数，条件为 `row + col == 2`。

源代码初版存在变量引用和遍历方式错误，而且没有处理两条对角线。改进版将 A、B 的状态分开保存，能够正确判断胜负。

```python
from typing import List


class Solution:
    def tictactoe(self, moves: List[List[int]]) -> str:
        rows = [[0] * 3 for _ in range(2)]
        cols = [[0] * 3 for _ in range(2)]
        diagonals = [0, 0]
        anti_diagonals = [0, 0]
```

```
        for index, (row, col) in enumerate(moves):
            player = index % 2
            rows[player][row] += 1
            cols[player][col] += 1

            if row == col:
                diagonals[player] += 1
            if row + col == 2:
                anti_diagonals[player] += 1

            if (
                rows[player][row] == 3
                or cols[player][col] == 3
                or diagonals[player] == 3
                or anti_diagonals[player] == 3
            ):
                return "A" if player == 0 else "B"

        return "Draw" if len(moves) == 9 else "Pending"
```

- 时间复杂度：$O(n)$，其中本题最多只有 9 步，也可视为 $O(1)$
- 空间复杂度：$O(1)$

数组题常用方法总结

1. 双指针

适用特征：数组两端或有序区间中的状态可以帮助排除答案。

- 11. 盛最多水的容器：移动较短边。
- 15. 三数之和：根据三数之和移动左右指针。
- 42. 接雨水：根据左右最大值决定处理哪一侧。

使用双指针前，需要明确：

1. 指针分别代表什么。
2. 什么条件下移动哪个指针。
3. 移动后为什么不会漏掉答案。

2. 哈希集合

适合需要快速判断“某个元素是否存在”的问题。

- 128. 最长连续序列 使用集合把查找复杂度从线性降低到平均常数级。
- 集合会去重，但没有可靠的数值顺序。
- 列表不可哈希，不能作为集合元素；不可变元组可以。

3. 栈

适合后进先出、撤销操作或依赖最近有效状态的问题。

- `682. 棒球比赛` 中，`append` 记录有效得分，`pop` 撤销最近得分。
- 使用 `scores[-1]`、`scores[-2]` 访问最近的有效记录，不应依赖原操作数组中的绝对下标。

4. 状态模拟

模拟题的关键是找出完整且最小的状态集合。

- 机器人：坐标和朝向。
- 井字棋：行、列、两条对角线以及当前玩家。
- 方向循环变化时，可以使用取模运算统一状态。

5. 从暴力解法到优化解法

本目录中的几次典型优化：

问题	暴力瓶颈	优化方式
盛最多水的容器	枚举所有边界组合	双指针排除无效组合
三数之和	三层循环	排序后固定一数，另外两数使用双指针
接雨水	每个位置重复求左右最大值	用变量保存遍历过程中的左右最大值
最长连续序列	排序或重复搜索	集合快速查找，只从序列起点出发
单调数列	两次排序	一次遍历维护两个可能状态

> 优化的核心通常是消除重复工作：保存已经得到的状态，或利用题目结构一次排除多个无效候选。